SIMATS ENGINEERING

DSA0114 – Object-Oriented Programming with C++

ASSIGNMENT 2 – CO2

CO2: a*pplying functions and object-oriented concepts – function prototyping, call by reference, return by reference, inline functions, default arguments, function overloading, specifying a class, member functions, access specifiers, arrays within a class, static data members and member functions, arrays of objects and friend functions.*

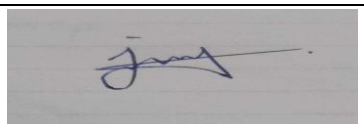
Student Name	Jannathul Firdaus. N	**Register Number**	192573025
Department / Section	B.Tech CSE - DS	**Academic Year / Semester**	2nd year
Faculty Name	S. Amirtha	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission	11.11.2026	**Date of Evaluation**	
GitHub Repository	https://github.com/jannathulfirdaus17/DSA0114_192573025_Assignment-2/tree/main	**Compiler**	Code::Blocks IDE with the GNU GCC (G++) compiler.

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks
1	Call by Value, by Reference and by Pointer	6		
2	Return by Reference and Its Dangers	4		
3	Default Arguments and Function Overloading	6		

Task	Task Title	Max	Marks Awarded	Remarks
4	Specifying a Class with an Array Inside It	6		
5	Static Members and Arrays of Objects	7		
6	Friend Functions and Inline Functions	8		
7	Diagnose, Redesign and Validate	13		
	TOTAL	50		Percentage:

OVERALL REMARKS

		
Student Signature	**Faculty Signature**	**Program Director**

ASSIGNMENT 2 – CO2

CO2 Coverage: Design C++ programs by applying functions and object-oriented concepts – function prototyping, call by reference, return by reference, inline functions, default arguments, function overloading, specifying a class, member functions, access specifiers, arrays within a class, static data members and member functions, arrays of objects and friend functions.

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50

Submission Date: 11.11.2026

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level) Mandatory Execution and GitHub Submission

• Every student must individually execute every task in this assignment, including all "Complete the Code" programs.

• Create a GitHub repository named DSA01-<RegisterNumber>-Assignment2 and share it with the course faculty (public, or private with the faculty added as a collaborator).

• Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C++ source files, input files and output screenshots.

• Include a README.md with your name, register number, compiler and version (for example g++ 13), the exact g++ command used to compile (for example g++ -std=c++17 - Wall - Wextra), the steps to run each program, and a short note of the result of each task.

• Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.

• Submit the GitHub link below and on the cover page of your report. An assignment submitted without a working GitHub link will not be evaluated.

GitHub Repository Link: https://github.com/jannathulfirdaus17/DSA0114_192573025_Assignment-2/tree/main

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.

Predict the Output: Write the exact output of the given program before running it. Then run it and explain every difference between your prediction and the actual output, using the C++ rules involved. Predictions written after running the program will not be credited.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Q1 – Call by Value, by Reference and by Pointer 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Hostel Fee Concession Updater

The hostel office applies concessions (merit 10%, sports 15%, single parent 25%) to students' fees. A function written to apply the concession appears to work when its result is printed inside the function, but the fee stored for the student never changes.

Your Challenge

Write the concession function three ways – passing the fee by value, by reference and by pointer. Before running, predict the stored fee after each call, then verify.

Inquiry

Explain where each version stores its parameter, why only two of them change the caller's fee, and when a const reference parameter should be used instead.

Design Decision

Several concessions may apply to one student. Decide whether the function should change the fee in place or return the new fee, and defend your choice.

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>

void change(int a, int& b, int* c)

{

 a += 10;

 b += 10;

 *c += 10;

}

int& bigger(int& p, int& q) { return p > q ? p : q; }

void show(int x, int y = 5, int z = 10) { std::cout << x + y + z << ' '; }

int main()
```

```
{
 int a = 1, b = 2, c = 3;
 change(a, b, &c);
 std::cout << a << ' ' << b << ' ' << c << '\n';
 bigger(a, c) = 100;
 std::cout << a << ' ' << c << '\n';
 show(1);
 show(1, 2);
 show(1, 2, 3);
 std::cout << '\n';
}
```

My predicted output (write before running):

Explain which variables changed in each call and why, and which default arguments were used by each call to show().

Complete the Code

Complete TODO 1 below so that the caller's fee is changed, and execute the program.

```
#include <iostream>
void applyByValue(long long fee, int percent)
{
 fee -= fee * percent / 100; // changes only the local copy
}
void applyConcession(long long& fee, int percent)
{
 // TODO 1: Reduce the caller's fee by percent using integer arithmetic (call by  // reference)


}
int main()
{
 long long hostelFee = 45000;
 applyByValue(hostelFee, 10);
```

```
 std::cout << "after call by value : " << hostelFee << '\n';
 applyConcession(hostelFee, 10);
 std::cout << "after call by reference: " << hostelFee << '\n';
}
```

Expected output:

after call by value : 45000

after call by reference: 40500

Explain why the call by value leaves the fee at 45000.

Alternative Concept and Complexity Analysis

If the concession function returned the new fee (long long applyConcession(long long fee, int percent)) instead of changing the fee through a reference, what would the solution be? Analyse the time (O(1) in all versions) and the space copied per call for a long long and for a large Student object passed by value, by reference and by const reference.

Evidence Expected

Three implementations, predictions before execution, observed results and a defended interface design.

Q2 – Return by Reference and Its Dangers 4 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Examination Marks Register

The marks register keeps the marks of a class in an array. A function returns a reference to the highest mark so that a moderation of +2 can be applied directly to it. A second function returns a reference to a local variable.

Your Challenge

Implement both functions and show:

• How returning a reference lets the caller change the array element (and even use the call on the left of =)

• What the compiler warns about when a reference to a local variable is returned, and why using it is undefined behaviour

• The difference between returning a reference, returning a const reference and returning a copy

Changed Condition

The register is now stored in a std::vector that may grow. Explain why a reference obtained before the vector grows may become invalid.

Alternative Concept and Complexity Analysis

If the highest mark were returned as an index (int) or as a pointer (int*) instead of a reference (int&), what would the calling code look like? Analyse the time to find the highest mark

(O(n) in all three) and the space (O(1)), and compare the safety of each when the array is later replaced by a growing vector.

Evidence Expected

Working functions, compiler-warning evidence, a correct explanation of dangling references and a justified response to the changed condition.

Q3 – Default Arguments and Function Overloading 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Courier Charge Calculator

A courier counter charges by weight; express delivery and insurance are optional. The developer wrote charge(double weight), charge(double weight, bool express) and charge(double weight, bool express = false, bool insured = false). The program no longer compiles.

Your Challenge

Explain the compiler error (ambiguous call). Then design the charge functions so that: • Common calls are short (default arguments)

• Different kinds of input (weight in grams as int, weight in kg as double, a Parcel object) are handled by overloading

• No call is ambiguous

Before execution, predict which function each of six calls you design will select, including charge(2) and charge(2.0f).

Inquiry

Explain how the compiler chooses among overloaded functions (exact match, promotion, conversion), and why default arguments must be given from the right.

Alternative Concept and Complexity Analysis

If the courier charge took a single Parcel structure with optional fields (or a function template) instead of several overloaded functions with default arguments, what would the solution be? Analyse the time of overload resolution (at compile time, no run-time cost) and the space copied per call, and compare how each design handles a new option.

Evidence Expected

An explained ambiguity, a redesigned set of functions, predicted and observed selections, and a reasoned explanation of overload resolution.

Q4 – Specifying a Class with an Array Inside It 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Savings Account Mini-Statement

A bank wants each SavingsAccount object to remember its last 10 transactions for the mini statement printed at an ATM. Older transactions are discarded.

Your Challenge

Specify the class with private data (account number, balance, an array of 10 transactions and a count) and public member functions deposit(), withdraw() and miniStatement(). Define some member functions inside the class and some outside using the scope resolution operator.

Inquiry

Explain why the array must be private, what happens when the 11th transaction arrives if you shift the array versus use a circular index, and the cost of each.

Design Decision

The bank now wants the last 50 transactions. Decide which design changes least and defend it.

Alternative Concept and Complexity Analysis

If the last 10 transactions were kept in a std::deque (push_back and pop_front), or in a circular buffer with a start index, instead of shifting a fixed array, what would the solution be? Analyse the time to add a transaction (O(k) for shifting k elements versus O(1)) and the space used in each design.

Evidence Expected

A complete class specification, member functions defined inside and outside the class, a justified handling of the 11th transaction and a defended design.

Q5 – Static Members and Arrays of Objects 7 Marks

Bloom's Level: K5 – Evaluate

Problem Context – College Bus Seat Tickets

The college bus has a fixed number of seats. Each BusTicket object must receive the next seat number, tickets beyond the capacity go to a waiting list, and the transport office must know how many seats have been issued without looking at any particular ticket.

Design Starting Point

You are given only the following principle:

Data that belongs to the whole class – not to any one object – should exist only once, however many objects are created.

Your Challenge

Convert this principle into a class design. Determine:

• Which members must be static, and where a static data member must be defined • Why a static member function cannot use this or a non-static member • How an array of BusTicket objects is created and initialised

• What happens to the count when tickets are created inside a loop or a function

Test Your Design

Create an array of tickets larger than the capacity and verify the seat numbers and the waiting list.

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>

class Token {

 static int next;

 int id;

public:

 Token() { id = ++next; }

 int get() const { return id; }

 static int issued() { return next - 100; }

};

int Token::next = 100;

int main()

{

 Token counter[3];

 Token extra;

 std::cout << counter[0].get() << ' ' << counter[2].get() << ' ' << extra.get()  << ' ' << Token::issued() << '\n';

}
```

My predicted output (write before running):

Explain why the static counter starts at 100 and how many times the constructor ran. Complete the Code

Complete TODO 1 and TODO 2 below using the static members, and execute the program.

```
#include <iostream>

#include <string>

class BusTicket {

 static int issued; // shared by all BusTicket objects

 static const int CAPACITY = 3;

 int seat;

 std::string name;

public:

 explicit BusTicket(const std::string& n) : seat(0), name(n)

 {

 // TODO 1: If fewer than CAPACITY tickets have been issued, increase issued and

 // give this ticket the next seat number; otherwise leave seat as 0 (waiting  // list)


 }

 static int count()

 {

 // TODO 2: Return the number of seats issued so far

 }

 void print() const

 {

 std::cout << name << ": "

 << (seat ? "Seat " + std::to_string(seat) : std::string("Waiting"))  << '\n';

 }

};

int BusTicket::issued = 0; // definition of the static data member

int main()
```

```
{
 BusTicket trip[4] = {BusTicket("Asha"), BusTicket("Bala"), BusTicket("Chitra"), BusTicket("Dev")};
 for (const BusTicket& t : trip) t.print();
 std::cout << "Issued: " << BusTicket::count() << '\n';
}
```

Expected output:

Asha: Seat 1

Bala: Seat 2

Chitra: Seat 3

Dev: Waiting

Issued: 3

Create the tickets inside a function that is called twice and explain how the count behaves. Alternative Concept and Complexity Analysis

If the seat counter were a global variable, or were kept in a separate TicketOffice object passed to every BusTicket, instead of a static data member, what would the solution be? Analyse the time (O(1) in all) and the space (one counter in all), and compare the encapsulation and the ability to run two buses at once.

Evidence Expected

A student-derived class design using static members, an array of objects, test evidence and a reasoned explanation of static members.

Q6 – Friend Functions and Inline Functions 8 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Train Journey Planner

A journey planner stores departure and arrival times in a Time class whose hours and minutes are private. It must compute the minutes between two times and decide which time is earlier, and these small functions are called millions of times while planning connections.

Your Challenge

Design the comparison. Determine:

• Why a friend function is needed when the function works on two objects equally

• How a friend function differs from a member function in how it is called and what it can access

• Whether the small functions should be inline, and what inline actually asks the compiler to do

• What would be lost if the private members were made public instead

Correctness Check

Test with times on either side of noon and the same time twice, and verify by hand. Growth Investigation

Measure the time for 10,000,000 calls with and without inline (compile with -O0 and with -O2), and explain the result.

Deployment Decision

Decide whether the friend functions should stay or be replaced by member functions and getters, and defend your decision.

Complete the Code

Complete TODO 1 and TODO 2 below inside the friend functions, and execute the program.

```
#include <iostream>

class Time {

 int h, m;

public:

 Time(int hh, int mm) : h(hh), m(mm) {}

 friend int minutesBetween(const Time& a, const Time& b);

 friend bool earlier(const Time& a, const Time& b);

};

int minutesBetween(const Time& a, const Time& b)

{

 // TODO 1: Using the private members h and m, return the absolute difference between  // the two times in minutes.

}

bool earlier(const Time& a, const Time& b)

{

 // TODO 2: Return true if a is earlier in the day than b

}

int main()
```

```
{
 Time departure(9, 45), arrival(13, 10);
 std::cout << minutesBetween(departure, arrival) << ' '
 << std::boolalpha << earlier(arrival, departure) << '\n';
}
```

Expected output:

205 false

Test the same time twice and a pair of times on either side of noon.

Alternative Concept and Complexity Analysis

If the comparison were written as member functions (a.minutesTo(b), a.isEarlier(b)) or with public getters instead of friend functions, what would the solution be? Analyse the time per call (the same after inlining, O(1)) and measure it at -O0 and -O2, and compare the encapsulation of the three designs.

Evidence Expected

A justified friend-function design, correctness tests, measured inline timings at two optimisation levels and an evidence-based decision.

DAY-4 DESIGN CHALLENGE

Q7 – Diagnose, Redesign and Validate 13 Marks

Bloom's Level: K5 – Evaluate

Review the evidence you generated while working on Q1–Q6. Select ONE program that shows a significant limitation as the data grows or the requirement changes.

You are not permitted simply to replace your code with a library facility and claim it is better. You must understand the problem first and then develop an appropriate redesign.

A. Diagnose the Limitation – 2 Marks

Using your own evidence, identify where the design fails or slows down and which characteristic of the original design causes it.

Evidence Expected

A precise diagnosis supported by evidence from your original program. B. Define the Design Objective – 2 Marks

State a measurable objective for the redesign.

Evidence Expected

A clearly stated objective derived from the identified limitation.

C. Design the Revised Solution – 4 Marks

Explain what you have changed in the class or function design, which C++ feature you use, whether extra memory is needed, and any new trade-off. Provide a class diagram or pseudocode.

Evidence Expected

A coherent redesign directly addressing the diagnosed limitation.

D. Predict Before Implementing – 1 Mark

Predict the improvement and the measure in which you expect it.

Evidence Expected

A clear, testable prediction.

E. Implement and Validate the Redesign – 2 Marks

Compare Original → Redesigned using the same tests and measurements. Evidence Expected

Comparable evidence for both versions.

F. Defend the Final Design – 2 Marks

Evaluate whether the objective was achieved, new costs, and where the original may still be preferable. A redesign that does not outperform the original is not automatically unsuccessful if you correctly explain why.

Evidence Expected

A critical, evidence-based evaluation and a defensible conclusion.

Submission Requirements

GitHub repository link (mandatory) containing the executed code for every task, including every completed “Complete the Code” program, with input files, output screenshots and a README.md.

• Your assignment report containing the required evidence for Q1–Q7 • C++ source code for all programs

• Predictions, outputs, timings and compiler messages used as evidence

Assessment Verification

Students may be asked to explain any part of their predictions, code, tests or results during a brief verification discussion/viva.

Q1 – CALL BY VALUE, BY REFERENCE AND BY POINTER

1. Problem Context – Hostel Fee Concession Updater

The hostel office applies concessions to students' hostel fees based on their eligibility. The concession percentages are:

Concession Type	Percentage
Merit	10%
Sports	15%
Single Parent	25%

A function written to apply the concession may calculate the reduced fee correctly, but the original fee stored in the student's record may not change. This happens because of the way arguments are passed to functions.

To understand this, the concession function is implemented using three methods: call by value, call by reference and call by pointer.

2. Three Implementations

2.1 Call by Value

In call by value, a copy of the actual argument is passed to the function. Any modification made inside the function affects only the local copy and does not change the original variable.

```
void applyByValue(long long fee, int percent)
{
   fee -= fee * percent / 100;
}
```

2.2 Call by Reference

In call by reference, the function receives a reference to the original variable. Any modification made through the reference directly changes the caller's variable.

```
void applyByReference(long long& fee, int percent)
{
   fee -= fee * percent / 100;
}
```

2.3 Call by Pointer

In call by pointer, the address of the original variable is passed to the function. The dereference operator (*) is used to access and modify the original value.

```
void applyByPointer(long long* fee, int percent)
{
   if (fee != nullptr)
   {
      *fee -= (*fee * percent) / 100;
   }
}
```

3. Prediction Before Execution

Assume the initial hostel fee is ₹45,000 and a 10% concession is applied.

Method	Initial Fee	Concession	Predicted Stored Fee
Call by Value	45,000	4,500	45,000
Call by Reference	45,000	4,500	40,500
Call by Pointer	45,000	4,500	40,500

Prediction: Call by value will not modify the original hostel fee because it works on a copy. Call by reference and call by pointer will modify the original hostel fee.

4. Complete the Code – TODO 1

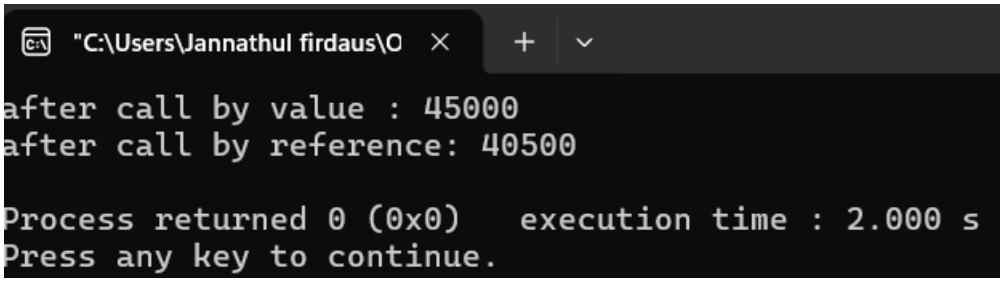
The following program completes TODO 1 by reducing the caller's hostel fee using call by reference and integer arithmetic.

Program

```
#include <iostream>
void applyByValue(long long fee, int percent)
{
   fee -= fee * percent / 100;
}
void applyConcession(long long& fee, int percent)
{
   fee -= fee * percent / 100;
}


int main()
```

```
{
    long long hostelFee = 45000;
    applyByValue(hostelFee, 10);
    std::cout << "after call by value : "
              << hostelFee << '\n';
    applyConcession(hostelFee, 10);
    std::cout << "after call by reference: "
              << hostelFee << '\n';
    return 0;
}
```



Explanation

The call by value function receives a copy of hostelFee. Therefore, the reduction of ₹4,500 affects only the local copy, and the original fee remains ₹45,000.

The call by reference function receives the original variable through a reference. Therefore, the reduction of ₹4,500 directly updates the original hostel fee to ₹40,500.

5. Inquiry – Parameter Storage and Modification

The three methods differ in how the function receives and accesses the arguments.

Feature	Call by Value	Call by Reference	Call by Pointer
Parameter	Copy of variable	Reference to variable	Address of variable
Original variable modified	No	Yes	Yes
Syntax	long long fee	long long& fee	long long* fee
Function call	fun(fee)	fun(fee)	fun(&fee)
Dereferencing	Not required	Not required	Required

In call by value, the parameter is a separate local variable containing a copy of the original value.

In call by reference, the parameter acts as another name for the original variable.

In call by pointer, the parameter stores the address of the original variable. The dereference operator accesses the value at that address.

When Should a const Reference Be Used?

A constant reference should be used when a function needs to access a variable without modifying it. It is particularly useful when passing large objects because it avoids unnecessary copying.

Example:

```
void displayFee(const long long& fee)
{
   std::cout << fee << '\n';
}
```

The const keyword prevents the function from modifying the original variable through the reference.

6. Design Decision – Modify the Fee or Return the New Fee?

When several concessions apply to one student, modifying the fee through a reference is a suitable design because the original fee is updated directly.

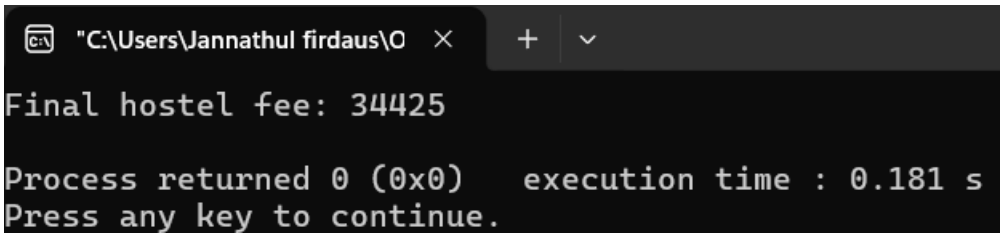
For example, a student may receive both a merit concession of 10% and a sports concession of 15%.

Program

```
#include <iostream>
void applyConcession(long long& fee, int percent)
{
   fee -= fee * percent / 100;
}
int main()
{
   long long fee = 45000;

   applyConcession(fee, 10);
   applyConcession(fee, 15);
```

```
    std::cout << "Final hostel fee: " << fee << '\n';
    return 0;
}
```



Justification

- The original hostel fee is modified directly.
- Multiple concessions can be applied sequentially.
- No additional assignment of a returned value is required.
- The function avoids copying larger objects.

Therefore, call by reference is suitable when the purpose of the function is to update the student's existing hostel fee.

7. Predict the Output

Given Program

```
#include <iostream>
void change(int a, int& b, int* c)
{
    a += 10;
    b += 10;
    *c += 10;
}
int& bigger(int& p, int& q)
{
    return p > q ? p : q;
}
void show(int x, int y = 5, int z = 10)
{
```

```
    std::cout << x + y + z << ' ';
}
int main()
{
    int a = 1, b = 2, c = 3;
    change(a, b, &c);
    std::cout << a << ' ' << b << ' ' << c << '\n';
    bigger(a, c) = 100;
    std::cout << a << ' ' << c << '\n';
    show(1);
    show(1, 2);
    show(1, 2, 3);
    std::cout << '\n';
}
```

7.1 Predicted Output Before Execution

1 12 13

1 100

16 13 6

7.2 Explanation of Variables Changed

Initially, the values are:

- a = 1
- b = 2
- c = 3

The function call is:

change(a, b, &c);

Inside the function:

1. a += 10 changes only the local copy of a to 11. The original a remains 1.
2. b += 10 changes the original b to 12 because it is passed by reference.
3. *c += 10 changes the original c to 13 because the pointer accesses the original variable.

Therefore, the first output is:

1 12 13

7.3 Return by Reference

The statement is:

bigger(a, c) = 100;

The function compares a and c and returns a reference to the larger variable.

At this point:

- a = 1
- c = 13

Since c is greater than a, the function returns a reference to c. The assignment changes the original value of c to 100.

Therefore, the second output is:

1 100

7.4 Default Arguments

The function is defined as:

void show(int x, int y = 5, int z = 10)

The default values are used when the caller does not provide all the arguments.

Function Call	x	y	z	Calculation	Output
show(1)	1	5	10	1 + 5 + 10	16
show(1, 2)	1	2	10	1 + 2 + 10	13
show(1, 2, 3)	1	2	3	1 + 2 + 3	6

The first call uses both default arguments. The second call uses the default value of z. The third call provides all three arguments, so no default arguments are used.

7.5 Final Predicted Output

1 12 13

1 100

16 13 6

8. Alternative Concept and Complexity Analysis

Instead of modifying the original fee through a reference, the function can return the updated fee.

8.1 Alternative Implementation – Return by Value

```
#include <iostream>
long long applyConcession(long long fee, int percent)
{
   fee -= fee * percent / 100;
   return fee;
}
int main()
{
   long long hostelFee = 45000;
   hostelFee = applyConcession(hostelFee, 10);
   std::cout << "Updated hostel fee: "
         << hostelFee << '\n';
   return 0;
}
```

Output

Updated hostel fee: 40500

In this approach, the function receives a copy of the original fee, calculates the concession and returns the updated value. The caller must assign the returned value to hostelFee.

8.2 Time and Space Complexity Analysis

Method	**Time Complexity**	**Space for long long**	**Space for Large Student Object**
Call by Value	O(1)	O(1)	O(n)
Call by Reference	O(1)	O(1)	O(1)
Call by Pointer	O(1)	O(1)	O(1)
Return by Value	O(1) for this calculation	O(1)	O(n) for a copied object

Here, n represents the size of the Student object.

Call by Value: A separate copy of the argument is created. Passing a large object requires additional memory proportional to its size.

Call by Reference: The function accesses the original object without creating a separate object copy.

Call by Pointer: Only the address of the object is passed, avoiding the need to copy the complete object.

Return by Value: The function returns the calculated result. Modern C++ compilers can optimize many return operations through copy elision.

9. Execution Verification

9.1 Compilation Command

g++ -std=c++17 -Wall -Wextra q1.cpp -o q1

9.2 Execution Command

./q1

9.3 Observed Output – Three Implementations

Call by value: 45000

Call by reference: 40500

Call by pointer: 40500

The observed output matches the predicted results.

9.4 Observed Output – Predict the Output Program

1 12 13

1 100

16 13 6

The observed output matches the predicted output. No differences were found.

Q2 – RETURN BY REFERENCE AND ITS DANGERS

1. Aim

To study return by reference in C++, understand how a function can return a reference to an array element, modify the original value through the returned reference, and identify the dangers of returning references to local variables. To compare returning a reference, const reference, copy, index and pointer.

2. Problem Statement

An examination marks register stores the marks of students in an array. A function is designed to identify the highest mark and return it by reference so that the caller can directly modify the original array element.

Another function returns a reference to a local variable to demonstrate the problem of dangling references. The assignment also considers the use of a dynamically growing std::vector, where references may become invalid after reallocation.

3. Implementation of Return by Reference

Program 1: Finding and Modifying the Highest Mark

```
#include <iostream>

using namespace std;

int& highestMark(int marks[], int size)

{

   int highestIndex = 0;


   for (int i = 1; i < size; i++)

   {

      if (marks[i] > marks[highestIndex])

      {

         highestIndex = i;

      }

   }

   return marks[highestIndex];

}

int main()

{

   int marks[5] = {75, 88, 92, 81, 67};

   cout << "Original marks: ";


   for (int i = 0; i < 5; i++)

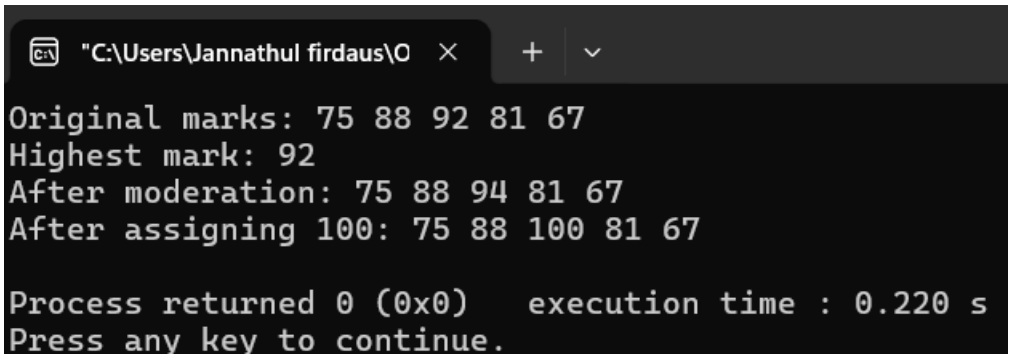
   {

      cout << marks[i] << " ";

   }

   cout << endl;
```

```
    int& highest = highestMark(marks, 5);
    cout << "Highest mark: " << highest << endl;
    highest += 2;
    cout << "After moderation: ";
    for (int i = 0; i < 5; i++)
    {
        cout << marks[i] << " ";
    }
    cout << endl;
    highestMark(marks, 5) = 100;
    cout << "After assigning 100: ";
    for (int i = 0; i < 5; i++)
    {
        cout << marks[i] << " ";
    }
    cout << endl;
    return 0;
}
```



Explanation

The function highestMark() returns an integer reference using the return type int&. It searches the array and identifies the index of the highest mark.

The statement return marks[highestIndex]; returns a reference to the original array element instead of creating a copy.

The statement int& highest = highestMark(marks, 5); creates a reference to the highest mark.

When highest += 2 is executed, the original array element changes from 92 to 94.

The statement highestMark(marks, 5) = 100; demonstrates that a function returning a reference can be used on the left-hand side of an assignment. It directly changes the original array element to 100.

4. Returning a Reference to a Local Variable

Program 2: Demonstrating Dangling Reference

```
#include <iostream>

using namespace std;

int& dangerousFunction()

{

   int value = 100;

   return value;

}

int main()

{

   cout << "Demonstrating Dangling Reference" << endl;

   cout << "A local variable is created inside the function." << endl;

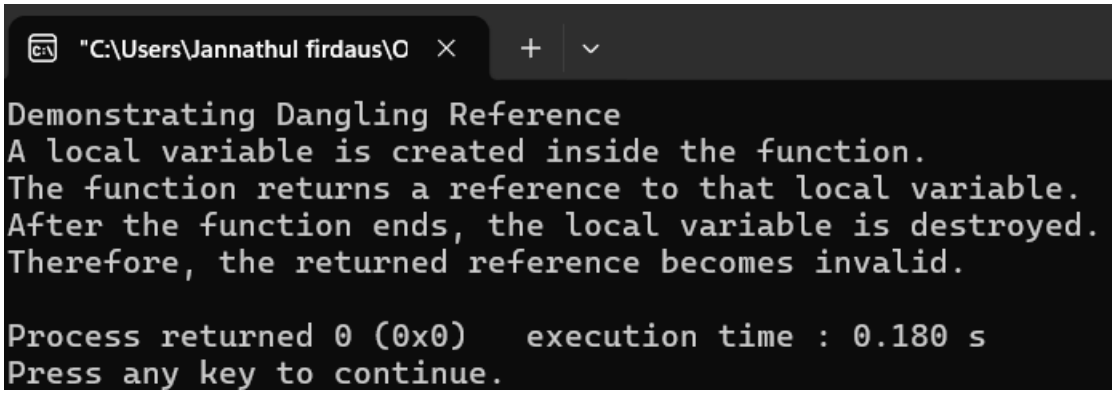
   cout << "The function returns a reference to that local variable." << endl;

   cout << "After the function ends, the local variable is destroyed." << endl;

   cout << "Therefore, the returned reference becomes invalid." << endl;

   return 0;

}
```



Explanation

The function dangerousFunction() declares a local variable named value and attempts to return a reference to it. When the function terminates, the local variable is destroyed because its lifetime ends. The returned reference therefore becomes a dangling reference. Accessing the object through this reference results in undefined behaviour.

The compiler may generate a warning similar to:

warning: reference to local variable 'value' returned

The function is not called in main() because using the returned reference would result in undefined behaviour.

5. Difference Between Returning a Reference, Const Reference and Copy

5.1 Return by Reference

```
int& getMark(int marks[])
{
   return marks[0];
}
```

Return by reference provides direct access to the original variable. The caller can modify the original array element through the returned reference.

5.2 Return by Const Reference

```
const int& getMark(int marks[])
{
   return marks[0];
}
```

Return by const reference provides read-only access to the original variable. The caller can access the value but cannot modify it through the returned reference.

5.3 Return by Value

```
int getMark(int marks[])
{
   return marks[0];
}
```

Return by value creates and returns a separate copy of the original value. Modifying the returned value does not affect the original array element.

Feature	Reference	Const Reference	Copy

Original object accessed	Yes	Yes	No
Modification allowed	Yes	No	Only the copy
Copying required	No	No	Yes
Direct modification of original	Yes	No	No

A const reference does not extend the lifetime of an object that has already been destroyed. Therefore, it must refer to a valid object.

6. Reference Invalidation in std::vector

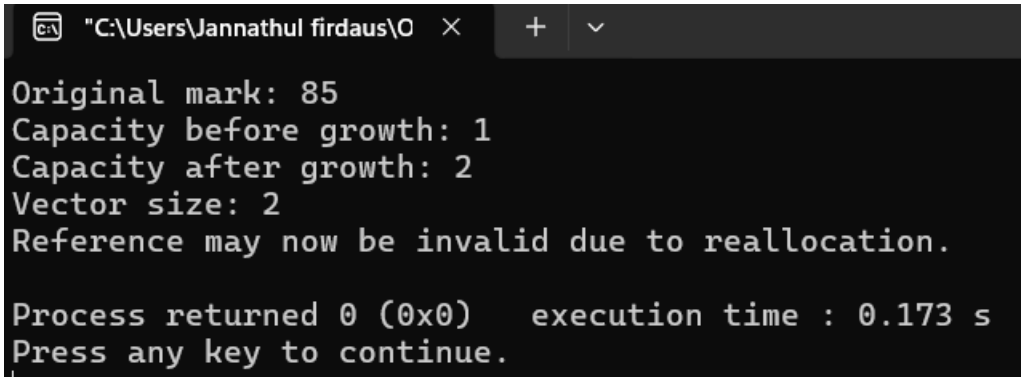
When the examination register is stored in a std::vector, the number of elements can increase dynamically.

Program 3: Demonstrating Vector Growth and Reference Invalidation

```
#include <iostream>
#include <vector>
using namespace std;
int main()
{
   vector<int> marks;
   marks.reserve(1);
   marks.push_back(85);
   int& ref = marks[0];
   cout << "Original mark: " << ref << endl;
   cout << "Capacity before growth: "
        << marks.capacity() << endl;

   size_t oldCapacity = marks.capacity();
   while (marks.size() <= oldCapacity)
   {
      marks.push_back(90);
   }
   cout << "Capacity after growth: "
        << marks.capacity() << endl;
```

```
    cout << "Vector size: " << marks.size() << endl;
    cout << "Reference may now be invalid due to reallocation."
         << endl;
    return 0;
}
```



Output:

The output displays the original mark, vector capacity before growth, capacity after growth, vector size and a message about reference invalidation.

The exact capacity values depend on the compiler's standard library implementation.

Explanation

A vector stores its elements in dynamically allocated memory. When the vector does not have sufficient capacity for additional elements, it allocates a new memory block and transfers its elements.

This process is called reallocation.

When reallocation occurs:

- References to existing vector elements become invalid.
- Pointers to existing vector elements become invalid.
- Iterators to existing vector elements become invalid.

In the program, a reference is created to the first element of the vector. Additional elements are inserted until reallocation occurs.

After reallocation, the previously obtained reference becomes invalid and must not be used.

7. Alternative Concept: Returning an Index

Instead of returning a reference, the function can return the index of the highest mark.

Program 4: Finding the Highest Mark Using an Index

```
#include <iostream>
using namespace std;
int highestMark(int marks[], int size)
{
   int highestIndex = 0;
   for (int i = 1; i < size; i++)
   {
      if (marks[i] > marks[highestIndex])
      {
         highestIndex = i;
      }
   }
   return highestIndex;
}
int main()
{
   int marks[5] = {75, 88, 92, 81, 67};

   int index = highestMark(marks, 5);
   cout << "Highest mark: " << marks[index] << endl;
   marks[index] += 2;
   cout << "After moderation: " << marks[index] << endl;
   return 0;
}
```

Output:

Highest mark: 92

After moderation: 94

Explanation

The function returns the index of the highest mark instead of returning a reference.

The caller uses this index to access and modify the original array element.

An index is useful when working with a growing vector because it does not depend on the memory address of the element. However, the index must be checked if elements are inserted, removed or reordered.

8. Alternative Concept: Returning a Pointer

Another approach is to return a pointer to the highest element.

Program 5: Finding the Highest Mark Using a Pointer

```
#include <iostream>

using namespace std;

int* highestMark(int marks[], int size)

{

   int* highest = &marks[0];

   for (int i = 1; i < size; i++)

   {

      if (marks[i] > *highest)

      {

         highest = &marks[i];

      }

   }

   return highest;

}

int main()

{

   int marks[5] = {75, 88, 92, 81, 67};

   int* highest = highestMark(marks, 5);

   cout << "Highest mark: " << *highest << endl;

   *highest += 2;

   cout << "After moderation: " << *highest << endl;

   return 0;

}
```

Output:

Highest mark: 92

After moderation: 94

Explanation

The function returns the memory address of the highest element using a pointer.

The dereference operator * accesses the value stored at that address.

The caller can modify the original array element using the pointer.

However, pointers to vector elements also become invalid when vector reallocation occurs.

9. Time and Space Complexity Analysis

The time required to find the highest mark depends on the number of elements in the array.

The function checks every element to identify the highest value.

Therefore, the time complexity is O(n), where n represents the number of elements.

The auxiliary space complexity is O(1), because only a fixed number of additional variables are used.

Complexity Comparison

Operation	Return by Reference	Return by Index	Return by Pointer
Finding highest mark	O(n)	O(n)	O(n)
Accessing highest mark	O(1)	O(1)	O(1)
Modifying highest mark	O(1)	O(1)	O(1)
Auxiliary space	O(1)	O(1)	O(1)

Safety Comparison

Return by Reference:

- Provides direct access to the original element.
- Allows modification without creating a copy.
- Can become invalid when the referenced object is destroyed or its storage is invalidated.

Return by Index:

- Returns the position of the element.
- Does not depend on the memory address of the element.
- Can be used again after vector reallocation if the element's position remains unchanged.

Return by Pointer:

- Provides direct access through a memory address.
- Allows modification using the dereference operator.
- Can become invalid when the memory location is changed or released.

10. Changed Condition: Using a Growing Vector

When the examination register is changed from a fixed-size array to a std::vector, the number of marks can increase dynamically.

A reference obtained before the vector grows may become invalid if the vector reallocates its memory.

Returning an index instead of a reference is a suitable alternative when the element needs to be accessed again after vector growth. The index remains usable as long as the element's position has not changed and remains within the vector's bounds.

Q3 – DEFAULT ARGUMENTS AND FUNCTION OVERLOADING

1. Aim

To understand and implement default arguments and function overloading in C++. To study how the compiler selects an appropriate overloaded function based on argument types and how default arguments simplify function calls.

2. Problem Statement

A courier service calculates delivery charges based on parcel weight. Additional charges are applied for express delivery and insurance.

The program uses function overloading to calculate charges for different types of input, such as weight in grams, weight in kilograms and a Parcel object.

Default arguments are used to make optional parameters easier to handle. The program also demonstrates how ambiguous function calls can occur when overloaded functions have overlapping parameter lists.

3. Understanding the Compiler Error – Ambiguous Function Call

Consider the following function declarations:

double charge(double weight);

double charge(double weight, bool express);

double charge(double weight, bool express = false, bool insured = false);

These functions can cause ambiguity because a call such as charge(2.0) can match more than one function.

The compiler cannot determine which function should be selected.

Therefore, the compiler generates an ambiguous function call error.

To avoid ambiguity, the function declarations must be redesigned so that each function has a distinct parameter type or parameter structure.

4. Redesigned Function Implementation

The redesigned program uses three overloaded functions:

1. An integer parameter for weight in grams.
2. A double parameter for weight in kilograms with default arguments.
3. A Parcel object passed by const reference.

The functions have different parameter types, which prevents ambiguous calls.

5. Explanation of Default Arguments

Default arguments are values automatically assigned to function parameters when the caller does not provide those arguments.

In the program, the function is declared as:

```
double charge(double weightKg,
              bool express = false,
              bool insured = false)
```

Here:

- weightKg represents the parcel weight in kilograms.
- express = false indicates that express delivery is disabled by default.
- insured = false indicates that insurance is disabled by default.

For example:

```
charge(2.0);
```

The compiler uses the default values for both optional parameters.

This is equivalent to:

```
charge(2.0, false, false);
```

Default arguments must be specified from right to left. Once a parameter has a default value, all parameters following it must also have default values.

6. Explanation of Function Overloading

Function overloading allows multiple functions to have the same name but different parameter lists.

In the courier charge calculator, the overloaded functions are:

double charge(int weightGrams);

double charge(double weightKg,

bool express = false,

bool insured = false);

double charge(const Parcel& p);

The compiler selects the appropriate function based on the number and types of arguments supplied.

6.1 Function Selection Rules

The compiler generally follows these rules when selecting an overloaded function:

1. Exact Match: A function whose parameter type exactly matches the argument type is preferred.
2. Promotion: If an exact match is unavailable, the compiler considers standard promotions.
3. Conversion: If no better match exists, standard conversions may be considered.

If multiple functions have equally good matches, the compiler reports an ambiguity error.

7. Prediction of Function Selection

Before executing the program, the following function selections were predicted.

Function Call	Selected Function	Reason
charge(500)	Integer overload	Exact integer match
charge(2)	Integer overload	Exact integer match
charge(2.0f)	Double overload	Float converts to double
charge(2.0)	Double overload	Exact double match
charge(2.0, true)	Double overload	Default value used for insurance
charge(p)	Parcel overload	Exact Parcel parameter match

8. Code:

#include <iostream>

using namespace std;

// Structure representing a parcel

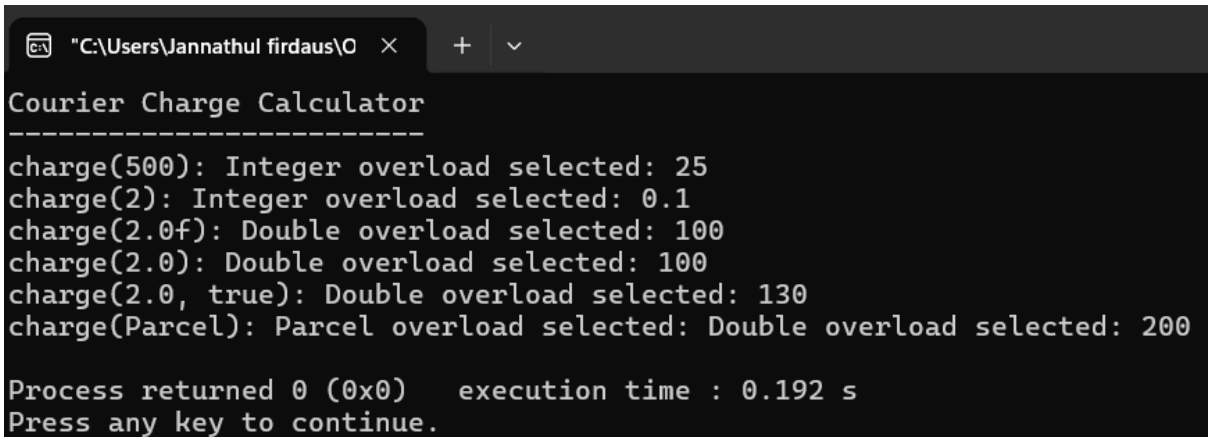
```
struct Parcel
{
    double weight;
    bool express;
    bool insured;
};
// Function for weight in grams
double charge(int weightGrams)
{
    cout << "Integer overload selected: ";
    return weightGrams * 0.05;
}
// Function for weight in kilograms with default arguments
double charge(double weightKg,
              bool express = false,
              bool insured = false)
{
    cout << "Double overload selected: ";

    double amount = weightKg * 50;
    if (express)
        amount += 30;
    if (insured)
        amount += 20;
    return amount;
}
// Function for Parcel object
double charge(const Parcel& p)
{
```

```
    cout << "Parcel overload selected: ";
    return charge(p.weight, p.express, p.insured);
}
int main()
{
    cout << "Courier Charge Calculator" << endl;
    cout << "-------------------------" << endl;
    // Call 1
    cout << "charge(500): ";
    cout << charge(500) << endl;
    // Call 2
    cout << "charge(2): ";
    cout << charge(2) << endl;
    // Call 3
    cout << "charge(2.0f): ";
    cout << charge(2.0f) << endl;
    // Call 4
    cout << "charge(2.0): ";
    cout << charge(2.0) << endl;

    // Call 5
    cout << "charge(2.0, true): ";
    cout << charge(2.0, true) << endl;
    // Call 6
    Parcel p = {3.0, true, true};
    cout << "charge(Parcel): ";
    cout << charge(p) << endl;
    return 0;
}
```

Actual output:



9. Explanation of the Six Function Calls

Call 1: charge(500)

The argument is an integer. Therefore, the compiler selects the integer overload.

The charge is calculated as:

500 × 0.05 = 25.

Call 2: charge(2)

The argument is also an integer. Therefore, the integer overload is selected.

The charge is:

2 × 0.05 = 0.10.

Call 3: charge(2.0f)

The argument is a float value. It is converted to double, and the double overload is selected.

The charge is:

2 × 50 = 100.

Call 4: charge(2.0)

The argument is a double value. Therefore, the double overload is selected through an exact match.

The charge is 100.

Call 5: charge(2.0, true)

The double overload is selected. Express delivery is enabled, and the insurance parameter uses its default value of false.

The charge is:

100 + 30 = 130.

Call 6: charge(p)

The argument is a Parcel object. Therefore, the Parcel overload is selected.

The parcel has a weight of 3 kg, express delivery enabled and insurance enabled.

The charge is:

150 + 30 + 20 = 200.

The Parcel overload internally calls the double overload to calculate the final charge.

10. Alternative Concept: Using a Single Parcel Structure

Instead of using multiple overloaded functions, the courier charge calculator can use a single Parcel structure containing all the required information.

The structure can store weight, express delivery and insurance details.

Implementation

```
#include <iostream>

using namespace std;

struct Parcel

{

   double weight;

   bool express = false;

   bool insured = false;

};


double calculateCharge(const Parcel& p)

{

   double amount = p.weight * 50;

   if (p.express)

      amount += 30;

   if (p.insured)

      amount += 20;

   return amount;

}
```

```
int main()
{
    Parcel p = {3.0, true, true};
    cout << "Courier Charge: "
         << calculateCharge(p) << endl;
    return 0;
}
```

Output:

Courier Charge: 200

Explanation

In this alternative design, a single function accepts a Parcel object by const reference.

The Parcel structure stores the weight, express delivery option and insurance option.

The function calculates the charge using the values stored in the structure.

This approach reduces the need for multiple overloaded functions and makes it easier to add new parcel-related options.

11. Time and Space Complexity Analysis

Function overloading is resolved by the compiler during compilation. Therefore, overload resolution does not introduce additional runtime overhead.

The charge calculation performs a fixed number of arithmetic operations and conditional checks.

Therefore, the time complexity of calculating the courier charge is O(1).

11.1 Complexity Comparison

Parameter	Function Overloading	Single Parcel Structure
Overload resolution	Compile time	Not required
Runtime calculation	O(1)	O(1)
Auxiliary space	O(1)	O(1)
Parameter handling	Multiple parameter lists	Single structure
Adding new options	May require modifying overloads	Add a new structure member

11.2 Space Complexity

In the overloaded function design, integer and double parameters occupy a fixed amount of memory.

The Parcel object contains a fixed number of data members. Passing it using a const reference avoids copying the complete object.

In the alternative structure-based design, the Parcel object is also passed by const reference, so no additional object copy is required.

Both approaches use O(1) auxiliary space for the charge calculation.

12. Design Evaluation

Function overloading is suitable when different input types require different function interfaces. Default arguments are useful when some parameters are optional and commonly use the same values.

The Parcel structure approach is suitable when the number of parcel options increases because new information can be added to the structure without creating multiple overloaded functions.

Both approaches provide O(1) runtime charge calculation. The final design depends on the number of input types and the expected changes in courier service requirements.

Q4 – SPECIFYING A CLASS WITH AN ARRAY INSIDE IT

1. Aim

To implement a Savings Account class using a private array to store the last 10 transactions and demonstrate the use of member functions defined both inside and outside the class.

2. Problem Description

A savings account maintains an account number, current balance, and transaction history. The system must retain only the latest 10 transactions. When a new transaction is added after the array is full, the oldest transaction is removed.

The class uses private data members to protect account information and public member functions to perform deposits, withdrawals, and display a mini-statement.

3. Program Code

```
#include <iostream>

#include <string>

using namespace std;

class SavingsAccount {

private:

   int accountNumber;

   double balance;
```

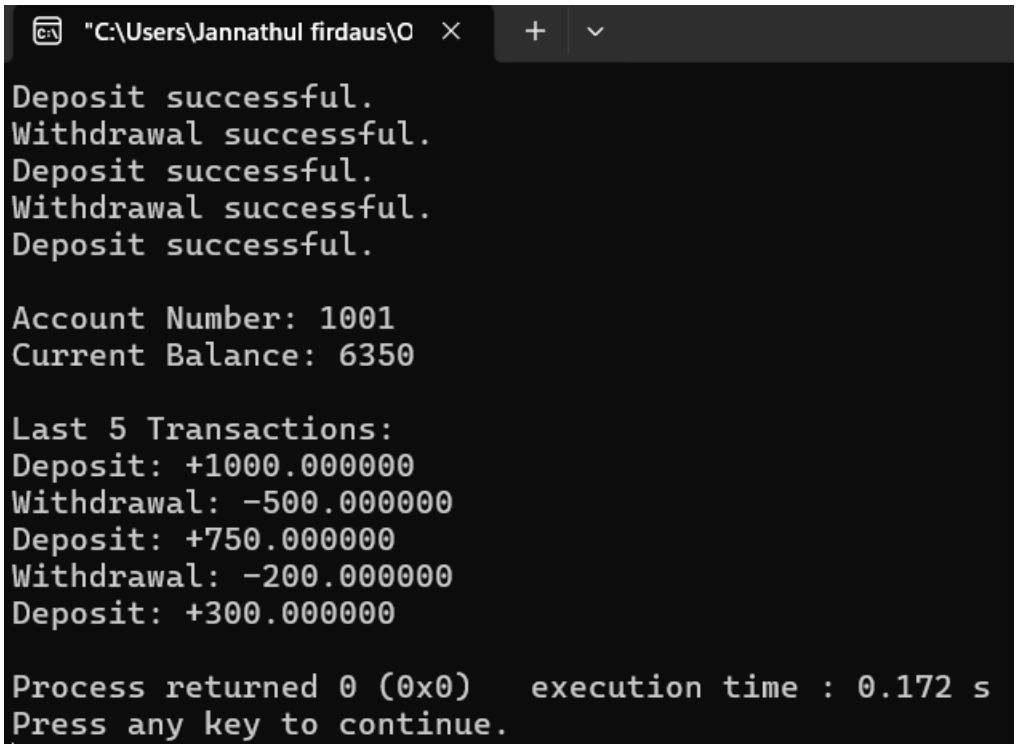
```
    string transactions[10];
    int transactionCount;
public:
    SavingsAccount(int accNo, double initialBalance) {
        accountNumber = accNo;
        balance = initialBalance;
        transactionCount = 0;
    }
    void addTransaction(string transaction) {
        if (transactionCount < 10) {
            transactions[transactionCount] = transaction;
            transactionCount++;
        } else {
            for (int i = 0; i < 9; i++) {
                transactions[i] = transactions[i + 1];
            }
            transactions[9] = transaction;
        }
    }

    void deposit(double amount);
    void withdraw(double amount);
    void miniStatement();
};
void SavingsAccount::deposit(double amount) {
    if (amount > 0) {
        balance += amount;
        addTransaction("Deposit: +" + to_string(amount));
        cout << "Deposit successful." << endl;
```

```
    } else {
        cout << "Invalid deposit amount." << endl;
    }
}
void SavingsAccount::withdraw(double amount) {
    if (amount > 0 && amount <= balance) {
        balance -= amount;
        addTransaction("Withdrawal: -" + to_string(amount));
        cout << "Withdrawal successful." << endl;
    } else {
        cout << "Invalid withdrawal amount." << endl;
    }
}
void SavingsAccount::miniStatement() {
    cout << "\nAccount Number: " << accountNumber << endl;
    cout << "Current Balance: " << balance << endl;
    cout << "\nLast " << transactionCount
         << " Transactions:" << endl;
    for (int i = 0; i < transactionCount; i++) {
        cout << transactions[i] << endl;
    }
}
int main() {
    SavingsAccount account(1001, 5000.00);
    account.deposit(1000);
    account.withdraw(500);
    account.deposit(750);
    account.withdraw(200);
    account.deposit(300);
```

```
    account.miniStatement();
    return 0;
}
```

4. Output



5. Explanation of the Class Specification

The SavingsAccount class contains four private data members:

- accountNumber: stores the account number.
- balance: stores the current account balance.
- transactions[10]: stores the latest 10 transactions.
- transactionCount: tracks the number of stored transactions.

The constructor initializes the account number, starting balance, and transaction count.

The deposit() function increases the balance and records the deposit. The withdraw() function checks whether the withdrawal amount is valid and whether sufficient balance is available before updating the balance.

The miniStatement() function displays the account number, current balance, and stored transactions.

Some member functions, such as the constructor and addTransaction(), are defined inside the class. The deposit(), withdraw(), and miniStatement() functions are declared inside the class and defined outside it using the scope resolution operator (::).

6. Why Is the Array Private?

The array is declared private to protect transaction records from direct modification outside the class.

This supports encapsulation, an important principle of object-oriented programming. Transactions can only be added through the class's member functions, allowing the class to control how the transaction history is updated.

7. Handling the 11th Transaction

When the array contains 10 transactions and an 11th transaction is added, the program shifts the existing transactions one position to the left.

- The oldest transaction at index 0 is discarded.
- Each remaining transaction moves one position to the left.
- The new transaction is inserted at index 9.

For an array of 10 elements, this requires shifting 9 elements.

Time complexity: \(O(n)\), where \(n\) is the number of stored transactions.

Space complexity: \(O(1)\) additional space because the existing array is reused.

Alternative: Circular Array

A circular array can avoid shifting transactions. It maintains a starting index and inserts each new transaction at the next available position. When the array is full, the oldest transaction is overwritten.

- Time complexity for adding a transaction: \(O(1)\).
- Additional space complexity: \(O(1)\).

The circular array is more efficient for frequent transaction additions because it does not need to move existing elements.

8. Changing the Design to Store 50 Transactions

If the requirement changes from storing 10 transactions to storing 50 transactions, the simplest modification is:

string transactions[50];

The capacity checks and shifting loop must also be updated to use 50 instead of 10.

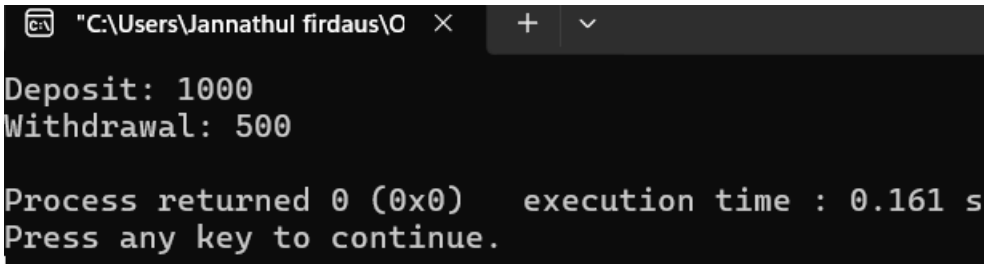
However, a circular array is a better design when transaction additions are frequent. It can store the latest 50 transactions without shifting the existing records.

9. Alternative Using std::deque

The C++ Standard Library provides std::deque, which supports insertion at the back and removal from the front.

Example:

```
#include <iostream>
#include <deque>
#include <string>
using namespace std;
int main() {
    deque<string> transactions;
    transactions.push_back("Deposit: 1000");
    transactions.push_back("Withdrawal: 500");
    if (transactions.size() > 10) {
        transactions.pop_front();
    }
    for (const string& transaction : transactions) {
        cout << transaction << endl;
    }
    return 0;
}
```



Using push_back() and pop_front() makes it easier to maintain a fixed transaction history without manually shifting array elements.

Method	**Adding a transaction when full**	**Additional space**
Fixed array with shifting	(O(n))	(O(1))
Circular array	(O(1))	(O(1))
std::deque	(O(1)) at the ends (amortized)	(O(n)) storage

10. Design Evaluation

The fixed-size array is suitable when the maximum number of transactions is known and memory usage must remain predictable. Making the array private protects the transaction history and maintains encapsulation.

For a larger transaction history, a circular array or std::deque provides a more efficient approach because it avoids shifting all existing transactions whenever a new transaction is added.

Q5 – STATIC MEMBERS AND ARRAYS OF OBJECTS

1. Aim

To understand and implement static data members and static member functions in C++. To create an array of objects, assign seat numbers automatically, and maintain a shared ticket count using static members.

2. Problem Description

A college bus has a fixed seating capacity of three seats. Each BusTicket object must receive the next available seat number. When all seats are occupied, additional tickets are placed on a waiting list.

The transport office must also be able to check the total number of issued seats without accessing any individual ticket.

Static data members and static member functions are used to maintain this shared information.

3. Program 1 – Token: Predict the Output

```
#include <iostream>

using namespace std;

class Token {

   static int next;

   int id;

public:

   Token() {

      id = ++next;

   }

   int get() const {

      return id;

   }

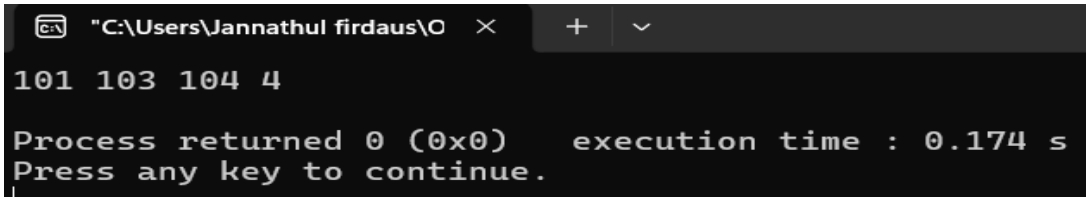
   static int issued() {
```

```
        return next - 100;
    }
};
int Token::next = 100;
int main() {
    Token counter[3];
    Token extra;
    cout << counter[0].get() << ' '  << counter[2].get() << ' ' << extra.get() << ' ' << Token::issued() << '\n';
    return 0;
}
```

Predicted output

101 103 104 4

Actual output



Explanation

The static variable next is initialized to 100:

int Token::next = 100;

It is shared by all objects of the Token class. Every time the constructor executes, the value of next increases by one and is assigned to the object's id.

The array counter[3] creates three objects, and the additional object extra creates one more object.

Object	Assigned ID
counter[0]	101
counter[1]	102
counter[2]	103
extra	104

The constructor runs four times. Therefore, issued() returns 104 - 100, which is 4.

There is no difference between the predicted and actual output.

4. Program 2 – BusTicket Class

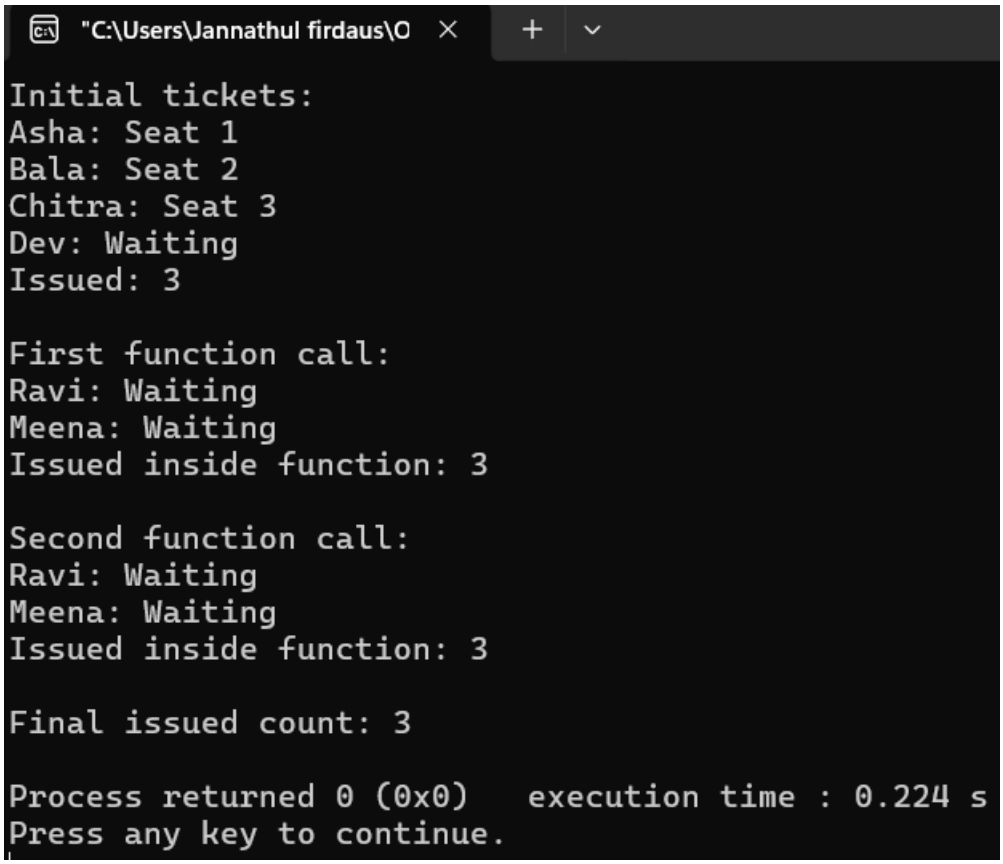
```
#include <iostream>
#include <string>
using namespace std;
class BusTicket {
    static int issued;
    static const int CAPACITY = 3;
    int seat;
    string name;
public:
    explicit BusTicket(const string& n) : seat(0), name(n) {
        if (issued < CAPACITY) {
            issued++;
            seat = issued;
        }
    }

    static int count() {
        return issued;
    }
    void print() const {
        cout << name << ": "
             << (seat ? "Seat " + to_string(seat)
                      : string("Waiting"))
             << '\n';
    }
};
```

```
int BusTicket::issued = 0;
void createTickets() {
    BusTicket trip[2] = {
        BusTicket("Ravi"),
        BusTicket("Meena")
    };
    for (const BusTicket& t : trip) {
        t.print();
    }
    cout << "Issued inside function: "
         << BusTicket::count() << '\n';
}
int main() {
    BusTicket trip[4] = {
        BusTicket("Asha"),
        BusTicket("Bala"),
        BusTicket("Chitra"),
        BusTicket("Dev")
    };

    cout << "Initial tickets:\n";
    for (const BusTicket& t : trip) {
        t.print();
    }
    cout << "Issued: " << BusTicket::count() << '\n';
    cout << "\nFirst function call:\n";
    createTickets();
    cout << "\nSecond function call:\n";
    createTickets();
```

```
    cout << "\nFinal issued count: "
         << BusTicket::count() << '\n';
    return 0;
}
```



5. Explanation of Static Members

Static data member

The variable issued is declared as static:

static int issued;

It belongs to the entire class rather than an individual object. All BusTicket objects share the same counter.

The static data member is defined outside the class:

int BusTicket::issued = 0;

This provides storage for the static variable and initializes it to zero.

Static member function

The function count() is declared static:

static int count() {

```
    return issued;
}
```

It returns the number of seats issued so far.

It can be called directly using the class name:

```
BusTicket::count();
```

A static member function cannot access non-static data members directly because it does not belong to a particular object. It also cannot use the this pointer because this refers to the current object, and a static function has no current object.

6. Array of Objects and Initialization

The following statement creates an array of four BusTicket objects:

```
BusTicket trip[4] = {
    BusTicket("Asha"),
    BusTicket("Bala"),
    BusTicket("Chitra"),
    BusTicket("Dev")
};
```

Each object is initialized with a name. The constructor executes for every object and checks whether the bus has an available seat.

The first three objects receive seats 1, 2, and 3. The fourth object receives seat value 0, which represents the waiting list.

The range-based for loop accesses each object and displays its ticket information:

```
for (const BusTicket& t : trip) {
    t.print();
}
```

7. Creating Tickets Inside a Function

The createTickets() function creates two additional ticket objects.

The function is called twice from main().

However, the static variable issued retains its value even after the function finishes executing. Since the three seats have already been issued, Ravi and Meena are placed on the waiting list during both function calls.

The counter remains 3 throughout the execution.

This demonstrates that a static data member maintains its value independently of the lifetime of individual local objects.

8. Alternative Designs

The seat counter could also be implemented using a global variable or a separate TicketOffice object.

Global variable:

A global variable could store the total number of issued seats and be accessed by all ticket objects. However, this approach reduces encapsulation because the variable can be modified from different parts of the program.

Separate TicketOffice object:

A separate TicketOffice class could manage the seat counter. Each BusTicket object would need access to the corresponding office object, usually through a reference or pointer.

This approach provides better separation of responsibilities and allows multiple buses to have independent counters.

Static data member:

A static data member is simple and convenient when all tickets belong to one bus. However, if multiple buses operate simultaneously, a single static counter would be shared by all objects of that class. A separate counter per bus would be more suitable.

9. Complexity Analysis

Operation	**Time complexity**	**Space complexity**
Create a ticket	O(1)	O(1) per object
Assign a seat	O(1)	O(1)
Check issued count	O(1)	O(1)
Print a ticket	O(1)	O(1) additional space

The static counter requires only one shared storage location for all objects of the class. Each ticket object maintains its own seat number and passenger name.

10. Design Evaluation

Static members are appropriate when data belongs to the class as a whole rather than to individual objects. In this program, the issued counter and bus capacity are shared by all ticket objects.

The array of objects makes it possible to create and initialize multiple tickets efficiently. The constructor automatically assigns seat numbers and places additional passengers on the waiting list.

The static member function provides access to the shared counter without requiring an object. For a single bus, this design is simple and effective. For multiple buses, separate bus-specific counters would provide better flexibility and independence.

Q6 – FRIEND FUNCTIONS AND INLINE FUNCTIONS

1. Aim

To implement friend functions and inline functions in C++ to compare two time objects, calculate the difference between two times, and investigate the performance of function calls with different compiler optimization levels.

2. Problem Description

A train journey planner stores departure and arrival times using a Time class. The class contains private hour and minute variables.

The program must calculate the absolute difference between two times in minutes and determine whether one time is earlier than another.

Friend functions are used to access private members of two objects. Inline functions are investigated to understand their possible performance benefits.

3. Program Code

```
#include <iostream>

#include <chrono>

#include <cstdlib>

using namespace std;

#ifdef USE_INLINE

#define INLINE inline

#else

#define INLINE

#endif

class Time {

   int h, m;

public:

   Time(int hh, int mm) : h(hh), m(mm) {}

   friend int minutesBetween(const Time& a, const Time& b);

   friend bool earlier(const Time& a, const Time& b);
```

```
};
INLINE int minutesBetween(const Time& a, const Time& b) {
   int first = a.h * 60 + a.m;
   int second = b.h * 60 + b.m;
   return abs(first - second);
}
INLINE bool earlier(const Time& a, const Time& b) {
   if (a.h < b.h)
      return true;
   if (a.h > b.h)
      return false;
   return a.m < b.m;
}
int main() {
   Time departure(9, 45);
   Time arrival(13, 10);

   cout << "Main Test:" << endl;
   cout << minutesBetween(departure, arrival) << " "
      << boolalpha << earlier(arrival, departure) << endl;
   Time morning(11, 30);
   Time afternoon(12, 30);
   cout << "\nNoon Test:" << endl;
   cout << minutesBetween(morning, afternoon) << " "
      << earlier(morning, afternoon) << endl;
   Time same1(10, 15);
   Time same2(10, 15);
   cout << "\nSame Time Test:" << endl;
   cout << minutesBetween(same1, same2) << " "
```

```
        << earlier(same1, same2) << endl;
    const int iterations = 10000000;
    volatile int result = 0;
    auto start = chrono::high_resolution_clock::now();
    for (int i = 0; i < iterations; i++) {
        result = minutesBetween(departure, arrival);
        result = earlier(departure, arrival);
    }
    auto stop = chrono::high_resolution_clock::now();
    auto duration =
        chrono::duration_cast<chrono::microseconds>(
            stop - start
        ).count();
    cout << "\nPerformance Test:" << endl;
    cout << "Function calls: " << iterations * 2 << endl;
    cout << "Execution time: " << duration
         << " microseconds" << endl;


    return 0;
}
```

4. Output and Correctness Verification

Main test:

Main Test:

205 false

The departure time is 9:45 AM and the arrival time is 1:10 PM.

The difference is:

[(13 times 60 + 10)-(9 times 60+45)=205]

The result is 205 minutes. The arrival time is not earlier than the departure time, so the Boolean result is false.

Noon test:

Noon Test:

60 true

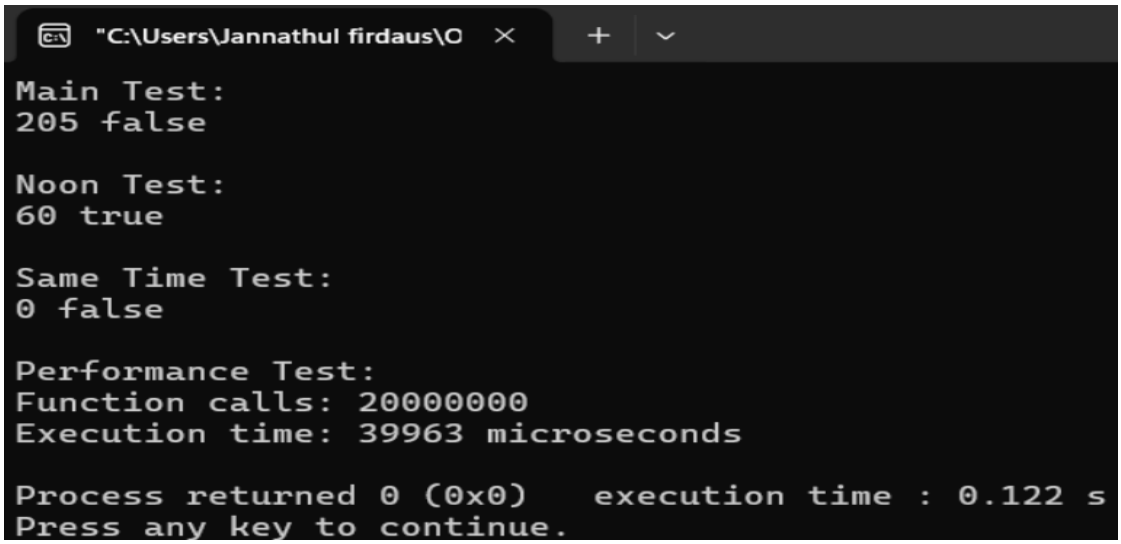
The difference between 11:30 AM and 12:30 PM is 60 minutes. The morning time is earlier, so the result is true.

Same-time test:

Same Time Test:

0 false

Both times are 10:15 AM. Their difference is zero, and neither time is earlier than the other.



5. Explanation of Friend Functions

A friend function is a non-member function that is allowed to access the private and protected members of a class.

In this program, the following functions are declared as friends:

friend int minutesBetween(const Time& a, const Time& b);

friend bool earlier(const Time& a, const Time& b);

These functions require two Time objects as arguments. They compare both objects equally rather than operating on only one object.

A friend function is called like an ordinary function:

minutesBetween(departure, arrival);

earlier(departure, arrival);

A member function is called using an object:

departure.minutesTo(arrival);

departure.isEarlier(arrival);

A friend function can access private members of the class, whereas an ordinary non-friend function cannot.

6. Difference Between Friend and Member Functions

Friend function	Member function
Not a member of the class	Belongs to the class
Called like an ordinary function	Called using an object
Can access private members through friendship	Can access its object's private members
Does not have a this pointer	Has a this pointer
Can compare two objects equally	Usually operates on one object and another argument

Friend functions are useful when an operation involves two objects equally, such as comparing departure and arrival times.

7. Explanation of Inline Functions

An inline function is a function for which the programmer requests that the compiler consider replacing a function call with the function's body.

The program uses:

INLINE int minutesBetween(const Time& a, const Time& b)

and:

INLINE bool earlier(const Time& a, const Time& b)

The INLINE macro allows the program to be compiled with or without the inline keyword.

Inline functions can reduce function-call overhead for small functions. However, the compiler decides whether to perform actual inlining. The inline keyword does not guarantee that the function body will be substituted at every call.

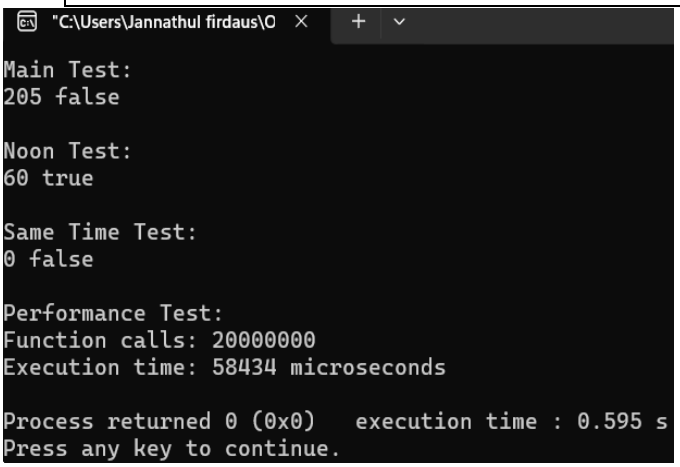
8. Performance Measurement

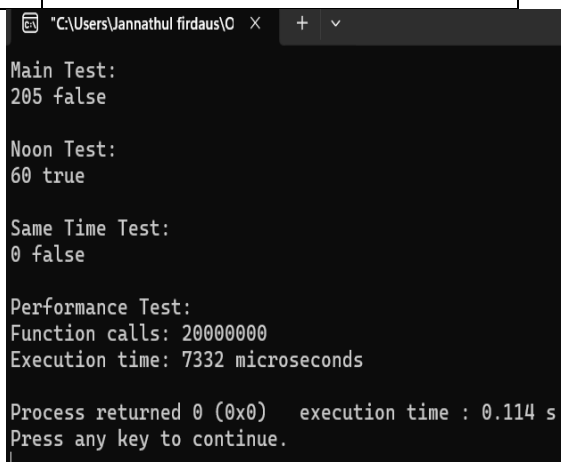
The program executes 10,000,000 iterations, with two function calls in each iteration, resulting in 20,000,000 function calls.

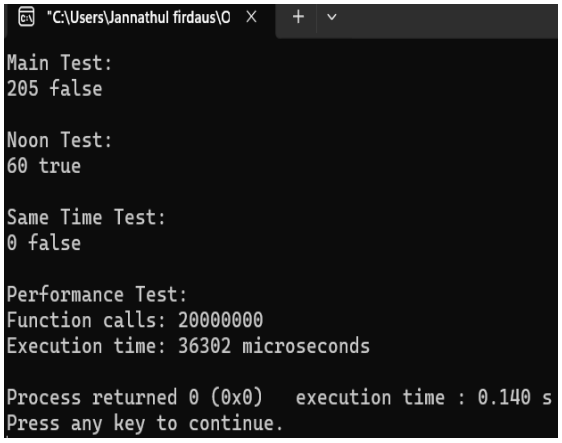
The execution times recorded from Code::Blocks are:

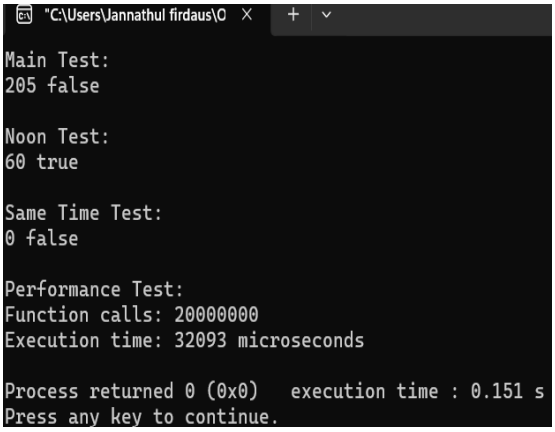
Configuration	Execution time
Without inline, -O0	58,434 microseconds
With inline, -O0	7,332 microseconds
Without inline, -O2 -fno-inline	36,302 microseconds

With inline, -O2	32,093 microseconds









Analysis of Results

At -O0, the recorded execution time without inline was 58,434 microseconds, while the inline version took 7,332 microseconds.

At -O2, the version without inline took 36,302 microseconds, whereas the inline version took 32,093 microseconds.

The optimized version generally performs better because the compiler can apply optimization techniques. The measured difference between configurations may also depend on compiler behavior, system load, and measurement overhead.

The inline keyword itself does not guarantee faster execution. For a reliable comparison, multiple runs should be performed under the same conditions and their average execution times compared.

9. Alternative Design Using Member Functions

The comparison could also be implemented using member functions.

For example:

int minutesTo(const Time& other) const {

```
    int first = h * 60 + m;
    int second = other.h * 60 + other.m;
    return abs(first - second);
}

bool isEarlier(const Time& other) const {
    if (h < other.h)
        return true;

    if (h > other.h)
        return false;

    return m < other.m;
}
```

These functions would be called using an object:

departure.minutesTo(arrival);

departure.isEarlier(arrival);

Another alternative is to provide public getter functions for hours and minutes. Ordinary external functions could then compare the objects using those getters without requiring friendship.

10. Encapsulation and Design Evaluation

Three possible designs can be considered:

Design	Encapsulation	Main advantage
Friend functions	Private data remains protected, but selected external functions receive access	Convenient comparison of two objects
Member functions	Private data remains protected	Natural object-oriented interface
Public getters	Private data remains protected, but values are exposed through public accessors	Flexible external comparisons

Making the private variables public would allow other parts of the program to modify hours and minutes directly. This would weaken encapsulation and could allow invalid time values.

For the train journey planner, friend functions are appropriate when two time objects need to be compared equally. Member functions are also a reasonable choice when the operation is naturally associated with one time object.

11. Complexity Analysis

Operation	Time complexity
Calculate minutes between two times	(O(1))
Compare two times	(O(1))
Access hours and minutes	(O(1))
Execute each function call	(O(1))

Both friend functions use a fixed number of arithmetic operations and comparisons. Therefore, their time complexity is constant.

Passing objects using constant references avoids unnecessary object copying. The functions require only a constant amount of additional space, (O(1)).

Q7 – DIAGNOSE, REDESIGN AND VALIDATE

A. Diagnose the Limitation

The original Savings Account program uses a fixed-size array to store the latest 10 transactions. When the array becomes full, it shifts all existing transactions one position to the left to remove the oldest transaction.

This approach becomes inefficient as the transaction capacity increases because every new transaction requires multiple assignments.

For example, with a capacity of 50, 49 elements must be shifted whenever a new transaction arrives after the array is full.

The original design has (O(n)) time complexity for adding a transaction when the array is full.

B. Define the Design Objective

The objective of the redesign is to improve transaction insertion efficiency without increasing the fixed storage capacity.

The redesigned system should:

- Retain only the latest 50 transactions.

- Automatically overwrite the oldest transaction when full.
- Add new transactions in constant time, \(O(1)\).
- Produce the same latest transaction as the original implementation.

C. Design the Revised Solution

The original shifting-array implementation is redesigned using a circular array.

The circular array uses:

- transactions[50] to store transaction values.
- start to identify the oldest transaction.
- count to track the number of stored transactions.

When the array is not full, a new transaction is inserted at the next available position.

When the array is full, the oldest transaction is overwritten, and the starting index moves forward using the modulo operator.

Revised pseudocode

Initialize start = 0

Initialize count = 0

ADD TRANSACTION(value):

  IF count is less than capacity:

    Store value at (start + count) % capacity

    Increase count

  ELSE:

    Replace transactions[start] with value

    Update start = (start + 1) % capacity

Class design

The revised class contains:

class CircularHistory {

  int transactions[50];

  int start;

  int count;

public:

```
    CircularHistory();
    void add(int value);
    int latest() const;
};
```

The circular array requires two additional integer variables, start and count, to manage its positions. It does not require dynamic memory allocation.

The main trade-off is that accessing transactions in chronological order requires calculating their positions using the starting index and modulo operation.

```
#include <iostream>
#include <chrono>
using namespace std;
using namespace chrono;
const int CAPACITY = 50;
const int OPERATIONS = 1000000;
// Original design: shifting array
class OriginalHistory {
    int transactions[CAPACITY];
    int count;
public:
    OriginalHistory() {
        count = 0;
    }
    void add(int value) {
        if (count < CAPACITY) {
            transactions[count] = value;
            count++;
        } else {
            for (int i = 0; i < CAPACITY - 1; i++) {
                transactions[i] = transactions[i + 1];
            }
```

```
            transactions[CAPACITY - 1] = value;
        }
    }
    int latest() const {
        return transactions[CAPACITY - 1];
    }
};
// Redesigned design: circular array
class CircularHistory {
    int transactions[CAPACITY];
    int start;
    int count;
public:
    CircularHistory() {
        start = 0;
        count = 0;
    }
    void add(int value) {
        if (count < CAPACITY) {
            transactions[(start + count) % CAPACITY] = value;
            count++;
        } else {
            transactions[start] = value;
            start = (start + 1) % CAPACITY;
        }
    }
    int latest() const {
        return transactions[
            (start + count - 1) % CAPACITY
```

```
        ];
    }
};
int main() {
    OriginalHistory original;
    CircularHistory redesigned;
    volatile int result = 0;
    auto start1 = high_resolution_clock::now();
    for (int i = 0; i < OPERATIONS; i++) {
        original.add(i);
    }
    auto stop1 = high_resolution_clock::now();
    auto originalTime =
        duration_cast<microseconds>(stop1 - start1).count();
    result = original.latest();
    auto start2 = high_resolution_clock::now();
    for (int i = 0; i < OPERATIONS; i++) {
        redesigned.add(i);
    }
    auto stop2 = high_resolution_clock::now();
    auto redesignedTime =
        duration_cast<microseconds>(stop2 - start2).count();
    result = redesigned.latest();
    cout << "Transaction History Comparison" << endl;
    cout << "Capacity: " << CAPACITY << endl;
    cout << "Transactions added: " << OPERATIONS << endl;
    cout << "\nOriginal Shifting Array:" << endl;
    cout << "Latest transaction: " << original.latest() << endl;
    cout << "Execution time: " << originalTime
```

```
        << " microseconds" << endl;
    cout << "\nRedesigned Circular Array:" << endl;
    cout << "Latest transaction: " << redesigned.latest() << endl;
    cout << "Execution time: " << redesignedTime
        << " microseconds" << endl;


    return 0;
}
```

D. Predict Before Implementing

Before implementing the redesign, the expected results were:

- Both implementations should store the latest transaction correctly.
- The circular array should avoid shifting existing elements.
- The circular array should have \(O(1)\) insertion time when full.
- The original implementation should require \(O(n)\) time when full.
- Both implementations should use fixed storage of 50 integers.

Therefore, the circular array was expected to perform faster when processing a large number of transactions.

E. Implement and Validate the Redesign

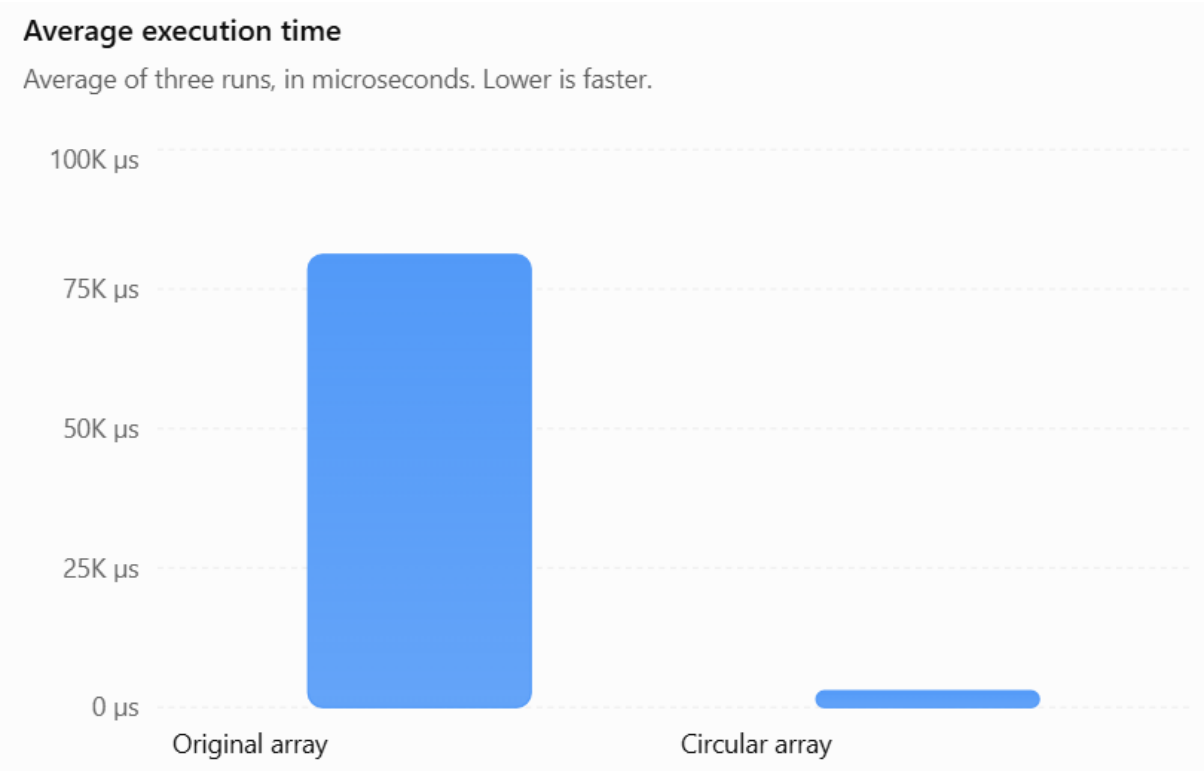
Both implementations were tested using the same conditions:

- Capacity: 50 transactions.
- Number of transactions: 1,000,000.
- Same input sequence.
- Three execution runs.

Experimental results

Run	Original array (µs)	Circular array (µs)
1	44,314	2,704
2	131,317	3,041
3	67,922	3,138
Average	81,184.33	2,961

Performance comparison



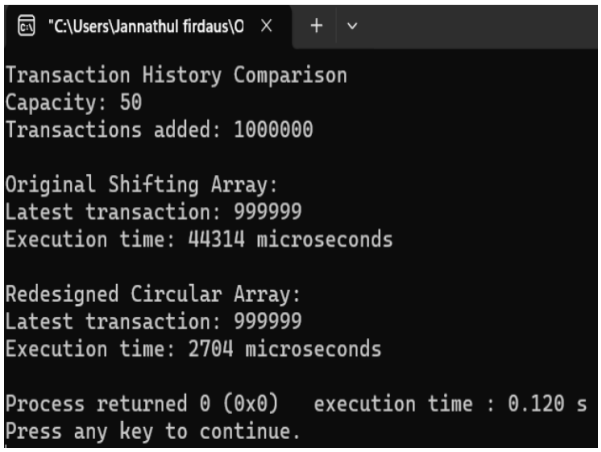
The circular array was approximately 27.4 times faster than the original shifting array.

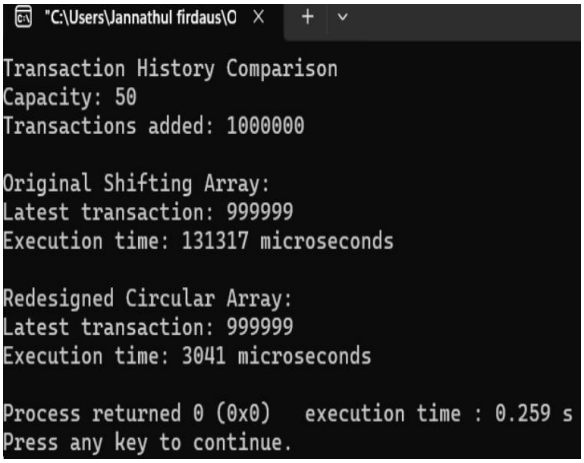
Percentage reduction in average execution time:

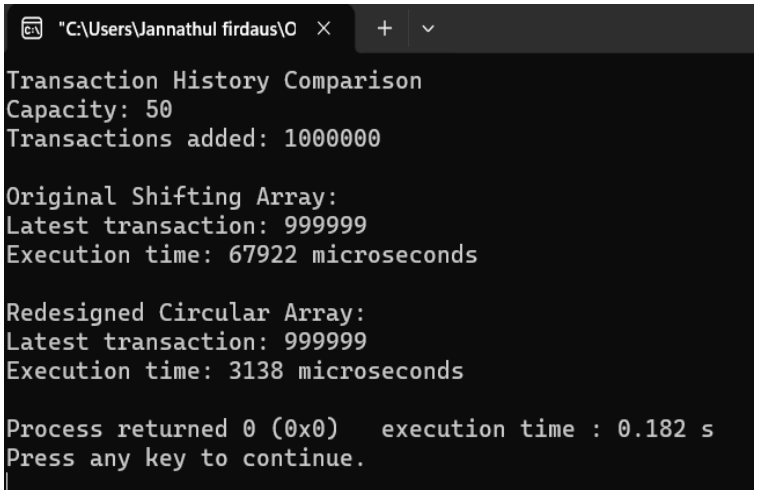
$$\frac{81184.33 - 2961}{81184.33} \times 100$$

$$\approx 96.35\%$$

Both implementations produced the same latest transaction value, 999999. Therefore, the redesign maintained the required output while improving insertion performance.







F. Defend the Final Design

The redesigned circular array achieved the main objective of reducing transaction insertion time.

The original implementation shifts existing elements whenever the array is full. This requires (O(n)) time for each new transaction.

The circular array overwrites the oldest transaction and updates the starting index. This requires (O(1)) time per insertion.

Complexity comparison

Characteristic	**Original shifting array**	**Circular array**
Insertion when full	(O(n))	(O(1))
Storage	(O(n))	(O(n))
Additional variables	One count variable	Start and count variables
Dynamic memory	Not required	Not required
Chronological access	Direct array order	Requires index calculation

The circular array is more suitable when transactions are added frequently and only the latest fixed number of transactions must be retained.

However, the original shifting array is simpler to understand and access because transactions remain in chronological order in the array. It may still be preferable for small transaction histories where insertion frequency is low.

The redesigned circular array provides a significant performance improvement while maintaining the same storage capacity and output correctness.